Review plan · for the teacher

Leading computing with few devices: unit check

Programming · Level 4 · Unit check

Time	Materials
45 min	Board, chalk, paper. No devices needed.

Learning goals

- Unplugged, then plugged
- One phone, many pupils
- Code on paper
- A scheme of work

Activity: Team quiz

1. Make teams of four, with the four roles
2. Read each question aloud; teams agree on one answer
3. Reveal the answer and ask one team to explain why
4. Swap roles after every question

Discussion

- Which idea will you put into practice first?
- Which question surprised you, and why?
- Who in your school will you share this with?

Exit ticket

- One change I will lead in our school's computing this term

Try it at school tomorrow
Choose one idea from this course (an unplugged-then-plugged pair, a one-phone rota, a paper-first lesson or a one-page term plan) and put it into practice this month. Then share what happened with a colleague.

Quiz sheet · for each team

Leading computing with few devices: unit check

Programming · Level 4 · Unit check

Team _____ Date _____

1 **What is the best reason to start a computing topic unplugged?**

- ◯ It replaces coding altogether
- ◯ Pupils meet the idea itself, without syntax in the way
- ◯ It saves the phones' batteries

2 **In pair programming, the driver and the navigator should swap roles every 5 to 10 minutes.**

- ◯ True
- ◯ False

3 **Paper strips: the program counts down from 5 to 1, then prints Lift off!. One strip doesn't belong.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

- ☐ `print("Lift off!")`
- ☐ `print(n)`
- ☐ `n = n + 1`
- ☐ `n = 5`
- ☐ `n = n - 1`
- ☐ `while n > 0:`

4 **During live coding, pupils should spot this mistake. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  light = input("Light? ")
2  if light = "red":
3      print("Stop")
```

5 **Match each paper exercise to what it shows you about a pupil.**

Parsons strips	• • • •	Whether they read code closely, and which misconceptions they hold
A trace table	• • • •	Whether they can read a whole program
Find the bug	• • • •	Whether they understand order and indentation
Predict the output	• • • •	How they think variables change

6 **A trace-table question. What does this program print?**

```
x = 2
for i in range(3):
    x = x * 2
print(x)
```

◯ 6 ◯ 8 ◯ 16 ◯ 12

7 **Which steps get phones ready for an offline lesson?**

☐ Charge the phones ☐ Download the lessons while online ☐ Print every screen

☐ Open one code step so Python is saved

8 **Fill the gaps: the program prints the biggest number eight bits can hold.**

```
print(2 ** _____ - _____)
```

Word bank: `7` `8` `0` `1`

9 **Put the steps of a live-coding moment in order.**

☐ Ask pupils to predict what it will do

☐ Write one line in front of the class

☐ Talk about any surprise

☐ Run it

10 **The binary cards in Python. Write value(up): up lists five 0s and 1s for the cards 16, 8, 4, 2 and 1 (1 means face up). Return the number shown.**

```
def value(up):
    cards = [16, 8, 4, 2, 1]
    # add up the face-up cards
    return 0
```

Write your program:

11 **A scheme of work kept only on one teacher's phone keeps computing going when that teacher leaves.**

◯ True ◯ False

Answers · for the teacher

Leading computing with few devices: unit check

Programming · Level 4 · Unit check

1 **Pupils meet the idea itself, without syntax in the way**

Unplugged lets every pupil grasp the idea first.

2 **True**

Frequent swaps mean both pupils do both jobs, and the faster typist doesn't take over.

3 **Top to bottom: 5, 3, X, 1, 4, 2**

```
n = 5
while n > 0:
    print(n)
    n = n - 1
print("Lift off!")
```

n goes down by 1 each turn until the condition is false.

4 **Line 2: `if light == "red":`**

Comparing needs ==. A single = stores a value, and Python stops with a SyntaxError: a mistake worth making on purpose in front of a class.

5 **Parsons strips — Whether they understand order and indentation**
A trace table — How they think variables change
Find the bug — Whether they read code closely, and which misconceptions they hold
Predict the output — Whether they can read a whole program

Each paper format tests a different part of understanding, and none needs a device.

6 **16**

x doubles three times: 4, 8, 16. "8" misses a turn (range(3) is three turns); "12" and "6" add instead of multiplying.

7 **Download the lessons while online; Open one code step so Python is saved; Charge the phones**

Downloaded lessons, Python saved once and charged batteries are all a lesson needs.

8

```
print(2 ** 8 - 1)
```

Eight bits make 2 ** 8 = 256 different values, from 0 to 255, so the biggest is 2 ** 8 - 1 = 255: the switches in the Bits and bytes lab.

9 **Top to bottom: 2, 1, 4, 3**

Predict before you run, every time: it keeps the whole class reading the code, and every surprise is a misconception to talk about.

10

```
def value(up):
    cards = [16, 8, 4, 2, 1]
    total = 0
    for i in range(5):
        if up[i] == 1:
            total = total + cards[i]
    return total
```

The unplugged activity, plugged in: walk along the cards and add the ones that are face up.

11 **False**

It leaves with them. A shared folder, trained colleagues and a club are what keep computing alive.